

Cairo Smart Traffic

1. Project Overview

Cairo Smart Traffic is a real-time AI monitoring system for road conditions across Greater Cairo. The platform watches 25 key locations continuously, predicts traffic up to 60 minutes ahead, detects anomalies, validates incoming data through a quality framework, and explains everything in plain language using AI summaries.

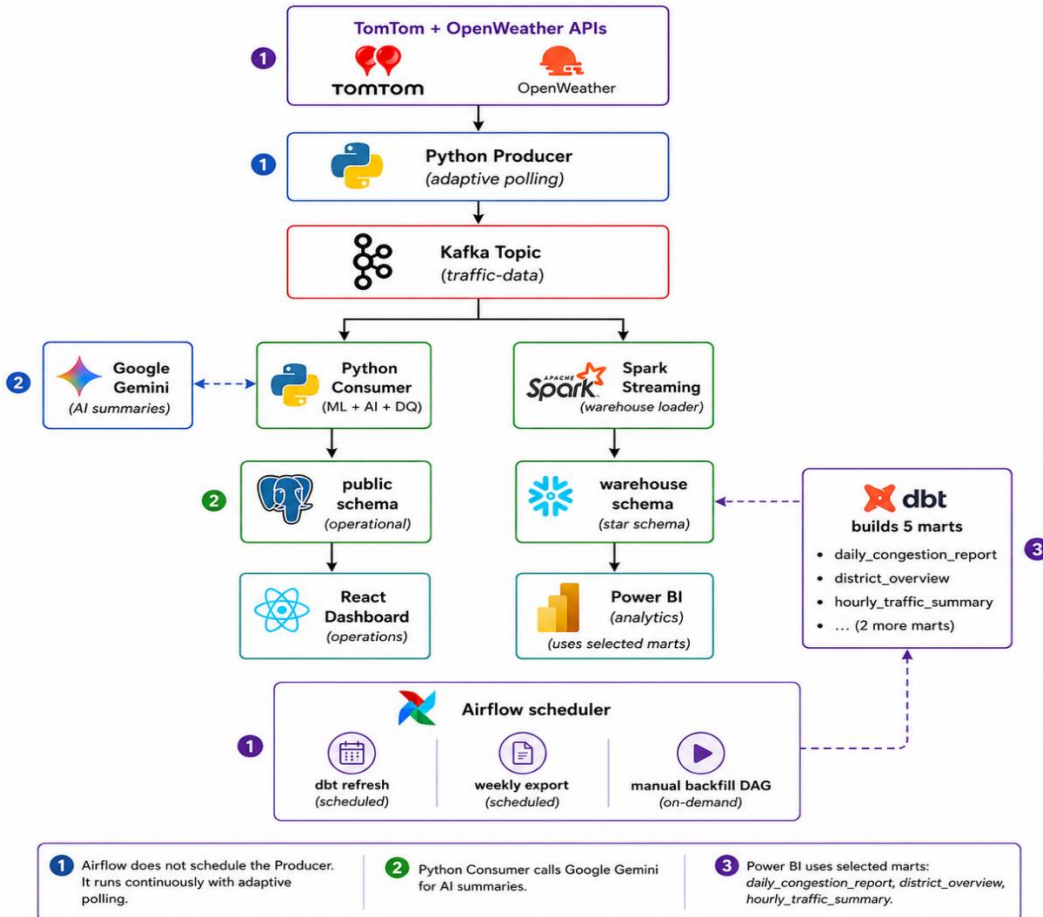
The project addresses a real operational gap. Cairo generates substantial traffic data through public APIs, but this data has historically been consumed reactively. Traffic operators respond to incidents after they happen. Planners describe past patterns rather than predicting future ones. We wanted to turn raw data streams into actionable intelligence.

2. What the System Does

The platform performs four functions simultaneously. It monitors current road conditions across 25 locations through a live operations dashboard. It predicts what conditions will look like at 20, 30, and 60 minutes ahead using machine learning models. It detects anomalies and surfaces them through real-time alerts. And it generates natural-language summaries of city traffic every few minutes through an AI integration with Google Gemini.

Three types of users benefit. Traffic operators use the real-time React dashboard for daily operations. Analysts and planners use Power BI for historical patterns and KPIs. Platform engineers monitor system health and data quality through dedicated dashboard pages.

3. Architecture



Data flows in a single direction from external APIs through to dashboards. The pipeline collects from TomTom Traffic and OpenWeather using a time-of-day-aware polling schedule that adjusts frequency based on Cairo local time. Apache Kafka serves as the message bus between collection and processing. Two consumers read from Kafka in parallel: a Python consumer handles real-time machine learning, AI summaries, and data quality validation, while Apache Spark Structured Streaming handles analytical loading into the data warehouse.

The storage layer has two schemas. An operational schema uses TimescaleDB hypertables to serve the real-time dashboard. An analytical schema implements a star schema with fact and dimension tables, on top of which a dbt project builds five mart tables for Power BI consumption. Apache Airflow orchestrates scheduled work through three DAGs.

The presentation layer also has two channels. A React dashboard provides real-time operations views, and a Power BI dashboard provides executive analytics. Both connect to the same underlying data infrastructure.

4. Machine Learning

The system uses two phases of machine learning. Real-time models look at each incoming reading and answer two questions instantly: what is the congestion level here, and is anything anomalous. Predictive models look at historical context per location and forecast what will happen 20, 30, and 60 minutes from now.

The real-time classifier is a Random Forest model that predicts congestion class. The anomaly detector is an Isolation Forest model paired with a rule-based post-filter that requires both a model signal and a corroborating domain indicator before raising an alert. The forecasting layer uses six LightGBM models — three horizons for speed regression and three for class classification. SHAP explainers accompany the classification models to provide per-prediction transparency.

5. AI Integration

Google Gemini generates natural-language summaries that appear at the top of the operations dashboard. A dedicated AI summary service gathers the latest state across all monitored locations, computes rollup metrics, sends a structured prompt to Gemini, and stores the response. The service runs decoupled from the main streaming pipeline, so any AI latency cannot block real-time processing.

6. Data Quality

Every incoming message passes through seven quality checks before reaching the machine learning pipeline. The framework validates required fields, speed sanity, free-flow speed sanity, closure-speed consistency, coordinate plausibility, reading freshness, and weather sanity. Each check produces a severity rating and contributes to a composite quality score. Failed messages are dropped from the pipeline, warnings pass through but are logged, and all results are persisted for observability through the dashboard's Data Quality page.

7. Storage Layer

The operational schema in PostgreSQL with TimescaleDB holds the working tables for the real-time dashboard, including raw events, predictions, anomalies, forecasts, AI summaries, and data quality events. The analytical warehouse schema implements a star schema with one fact table (fact_traffic) and three dimensions (dim_location, dim_time, dim_weather).

On top of the warehouse, a dbt project transforms raw fact data into five mart tables: daily_congestion_report (one row per road per day), district_overview (district-level rollups), hourly_traffic_summary (hourly patterns across all locations), rush_hour_analysis (morning and evening peak comparisons), and weather_impact_analysis (correlations between weather and

traffic). A staging view (`stg_traffic_enriched`) joins the fact table with all dimensions to simplify mart queries.

8. Orchestration

Three Airflow DAGs are defined and registered with the scheduler. The `dbt_hourly` DAG is designed to refresh the analytical marts on an hourly cadence. The `powerbi_weekly_export` DAG produces a weekly SQL dump of the warehouse schema for offline Power BI workflows. The `forecast_backfill` DAG provides cold-start recovery, used when the system first deploys or recovers from an extended outage.

9. Deliverables

We delivered a complete working system, technical documentation, and a public GitHub repository. The full stack runs in Docker with infrastructure containers orchestrated by `docker-compose`.

10. Challenges and Solutions

1. Challenge 1: TomTom API Rate Limit

Problem: TomTom's free tier allows 2,500 API calls per day. A naive fixed-interval polling strategy at 3-minute intervals across 25 locations would consume 12,000 calls per day — nearly five times the quota.

Solution: We implemented adaptive polling in `data_generator/adaptive_polling.py`. The polling interval varies by time of day in Cairo: 10 minutes during morning and evening rush hours when traffic changes rapidly, 20 minutes during midday and evening when changes are slower, and 30 minutes overnight. Daily total stays around 2,000 API calls, well within quota, while preserving granularity when it matters most. Comments in the file document the original schedule (5/10/15 min) and the doubled current schedule.

2. Challenge 2: Forecast Cold Start

Problem: The forecasting models require historical context per location — specifically, lag features going back 24 hours. On a fresh deployment, the consumer has no buffer history, so forecasts are unavailable until enough new data accumulates naturally.

Solution: We built a backfill workflow that pulls historical readings from the `raw_events` table and pre-populates the in-memory location buffers. Wrapped in an Airflow DAG (`forecast_backfill`) for operator-friendly triggering. The DAG is intentionally manual-trigger only because running it while the consumer is active would corrupt live buffer state. This is the kind of operational tool real production systems include.

3. Challenge 7: Data Quality at the Source

Problem: TomTom occasionally returned inconsistent data — closure flags set on roads where traffic was clearly flowing, speeds outside physically plausible ranges, and stale readings during their API hiccups. Without a quality layer, garbage flowed straight into the ML models.

Solution: We built a seven-check data quality framework in `stream_processor/data_quality.py`. Each message is scored against rules for required fields, speed sanity, free-flow sanity, closure-speed consistency, coordinate plausibility, freshness, and weather sanity. Failed messages are dropped; warnings pass through but are logged. All results write to a TimescaleDB hypertable, and the dashboard's Data Quality page surfaces trends in real time.

11. Team Responsibilities

Team Member	Responsibilities
Shahd	Built the real-time consumer pipeline and AI integration, including the Python consumer for ML inference, data quality validation, AI summary integration with Gemini, and the React operations dashboard.
Omnya	Built the Power BI dashboard with 13 visuals and the analytical view used by planners and executives.
Abdelrahman	Built the Spark Streaming consumer that loads the warehouse and developed the complete dbt project containing six models.
Ebtihal	Set up Apache Airflow and defined the three DAGs for mart refresh, weekly export, and cold-start backfill.
Aya	Designed the data warehouse, modeling the star schema with its fact table and three dimensions.

The team coordinated through GitHub for version control, daily messaging for synchronization, and shared documentation workspaces.